



CHAPTER 4

Building Good Training Datasets

Data Preprocessing

How the quality of a dataset shapes what a machine learning algorithm can learn

What We Cover

- | | | |
|-----------|----------------------------------|--|
| 01 | Handling Missing Data | Detecting, removing, and imputing missing values |
| 02 | Categorical Data | Ordinal vs. nominal features, encoding strategies |
| 03 | Train / Test Partitioning | Splitting data for unbiased evaluation |
| 04 | Feature Scaling | Normalization and standardization |
| 05 | Feature Selection | Regularization, sequential selection, random forests |

Data Quality Drives Model Quality

The **quality of the data** and the **amount of useful information** it contains are the key factors that determine how well a machine learning algorithm can learn. Before feeding data to a model, it must be examined and preprocessed.

1

Missing Values

Blanks or placeholders (NaN, NULL) that most algorithms cannot handle directly.

2

Categorical Data

Text labels — ordinal or nominal — that must become numeric before training.

3

Irrelevant Features

Noise and redundancy that inflate variance and hurt generalization.

01

Dealing With Missing Data

Identify, remove, or impute — three complementary strategies

SECTION 01

Identifying Missing Values in Tabular Data

Missing values commonly appear as **blank cells or placeholder strings such as NaN** (“not a number”) or NULL, a common indicator of unknown values in relational databases. Most computational tools cannot handle them, or will **produce unpredictable results** if they are simply ignored.

A	B	C	D
1.0	2.0	3.0	4.0
5.0	6.0	NaN	8.0
10.0	11.0	12.0	NaN

- **isnull()**
Returns a boolean mask flagging each cell as numeric or missing.
- **sum()**
Chained onto isnull() to count missing values per column.
- **values**
Exposes the underlying NumPy array of a DataFrame for scikit-learn.

Two Ways to Handle Missing Values

REMOVE

Eliminate Examples or Features

Drop rows or columns containing missing values entirely — `df.dropna(axis=0)` removes rows; `axis=1` removes columns.

- + Simple and fast to apply
- Risks losing too many samples
- Risks losing valuable feature columns

ESTIMATE

Impute Missing Values

Estimate missing entries from other training examples using interpolation, most commonly mean imputation via scikit-learn's **SimpleImputer** or pandas' **fillna()**.

- + Preserves all training examples
- + strategy: mean, median, or mode
- + `most_frequent` suits categorical columns

02

Handling Categorical Data

Ordinal features carry order; nominal features do not

SECTION 02

Ordinal vs. Nominal Features

Categorical data splits into two kinds. Ordinal features can be sorted or ordered; nominal features carry no inherent order.

ORDINAL

T-Shirt Size

A defined order exists between categories:



Mapped to integers, e.g. $XL = L + 1 = M + 2$

```
size_mapping = {'XL': 3, 'L': 2, 'M': 1}
```

NOMINAL

T-Shirt Color

No meaningful order — red is not “larger” than blue:



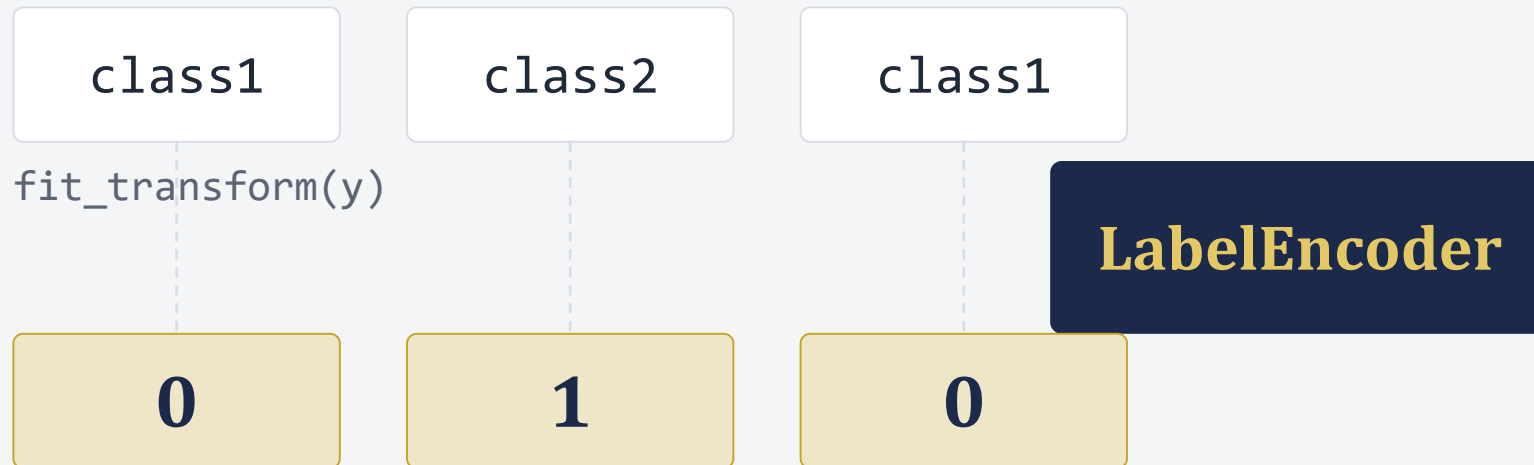
Requires one-hot encoding, not integer mapping

```
pd.get_dummies(df[['color']])
```

SECTION 02

Encoding Class Labels

Class labels are not ordinal, so it doesn't matter which integer is assigned to which string label. scikit-learn's **LabelEncoder** makes this a one-line operation, and `inverse_transform` reverses it.



`inverse_transform(y)` reverses the mapping back to original string labels

SECTION 02

One-Hot Encoding for Nominal Features

Integer-mapping a nominal feature like color would wrongly imply $\text{blue} > \text{green} > \text{red}$. One-hot encoding instead creates one binary dummy feature per category.

Before

color
green
red
blue

`get_dummies()`



After

blue	green	red
0	1	0
0	0	1
1	0	0

Caution: Multicollinearity

One-hot encoding introduces correlated columns, which can be a problem for methods that require matrix inversion. **Drop one redundant column (`drop_first=True`) to remove the correlation** — no information is lost, since $\text{blue}=0$ and $\text{green}=0$ already implies red.

03

Partitioning Train & Test Sets

Withholding data is the only way to estimate generalization

SECTION 03

Splitting Data for Unbiased Evaluation

Comparing predictions to true labels in a held-out test set gives an unbiased estimate of performance before a model is deployed. scikit-learn's `train_test_split` handles this randomly, and `stratify=y` preserves class proportions in both splits.

X_train / y_train — 70%

X_test / y_test — 30%

178 Wine examples → 13 chemical-property features, 3 cultivar classes

60 : 40

Smaller datasets

70 : 30

Common default

80 : 20

Common default

90:10 / 99:1

Large datasets (100k+)

04

Bringing Features Onto the Same Scale

Most learning algorithms behave better when features share a scale

SECTION 04

Why Feature Scaling Matters

Decision trees and random forests are scale-invariant — nearly everything else is not. A feature measured 1–10 and another measured 1–100,000 will dominate distance-based and gradient-based algorithms alike.

Adaline / Gradient Descent

The squared-error cost function is dominated by the larger-scale feature, so optimization mostly chases its errors.

K-Nearest Neighbors

Euclidean distance between examples is dominated by whichever feature axis has the largest range.

Scale-Invariant Exceptions

Decision trees and random forests split on thresholds per feature, so they don't require scaling at all.

Normalization vs. Standardization

Min-Max Normalization

Rescales features to a bounded [0, 1] range using the column's min and max. Useful when values must sit in a fixed interval.

MinMaxScaler()

$$x_{norm}^{(i)} = \frac{x^{(i)} - x_{min}}{x_{max} - x_{min}}$$

Input Standardized

$x^{(i)}$ is a particular example, x_n is the smallest value in a feature column, and x_m is the largest value.

Standardization

Centers features at mean 0 with unit variance, matching a standard normal distribution. Preserves outlier information; preferred for gradient descent, logistic regression, and SVM.

StandardScaler()

$$x_{std}^{(i)} = \frac{x^{(i)} - \mu_x}{\sigma_x}$$

μ_x is the sample mean of a particular feature column, and σ_x is the corresponding standard deviation

Input	Standardized	Min-max normalized
0.0	-1.46385	0.0
1.0	-0.87831	0.2
2.0	-0.29277	0.4
3.0	0.29277	0.6
4.0	0.87831	0.8
5.0	1.46385	1.0

we fit the scaling only once—on the training data—and use those parameters to transform the test dataset or any new data point.

```
from sklearn.preprocessing import MinMaxScaler  
mms = MinMaxScaler()  
X_train_norm = mms.fit_transform(X_train)  
X_test_norm = mms.transform(X_test)
```

```
from sklearn.preprocessing import StandardScaler  
stdsc = StandardScaler()  
X_train_std = stdsc.fit_transform(X_train)  
X_test_std = stdsc.transform(X_test)
```

SECTION 04

Other Scaling Options in scikit-learn

RobustScaler

Operates on each feature independently: removes the median and scales by the 1st and 3rd quartile (25th/75th percentile) instead of mean and standard deviation.

Best For

Small datasets that contain many outliers, or algorithms prone to overfitting on extreme values.

Effect

Extreme values and outliers become less pronounced after transformation, without discarding them entirely.

05

Selecting Meaningful Features

Reducing dimensionality to fight overfitting and high variance

SECTION 05

When a Model Overfits

A model that performs much better on training data than on test data has high variance — it is too complex for the amount of training data available. Four common remedies:

- 1 Collect more training data
- 2 Introduce a penalty for complexity via regularization
- 3 Choose a simpler model with fewer parameters
- 4 **Reduce the dimensionality of the data via feature selection**

SECTION 05

L2 vs. L1 Regularization — A Geometric View

Regularization adds a penalty term to the cost function that shrinks weights toward zero. The shape of that penalty budget determines where the optimum lands.

L2 Regularization

Penalty is the sum of squared weights — a circular (spherical) budget.

$$L2: \quad \|\mathbf{w}\|_2^2 = \sum_{j=1}^m w_j^2$$

L1 Regularization

Penalty is the sum of absolute weights — a diamond-shaped budget.

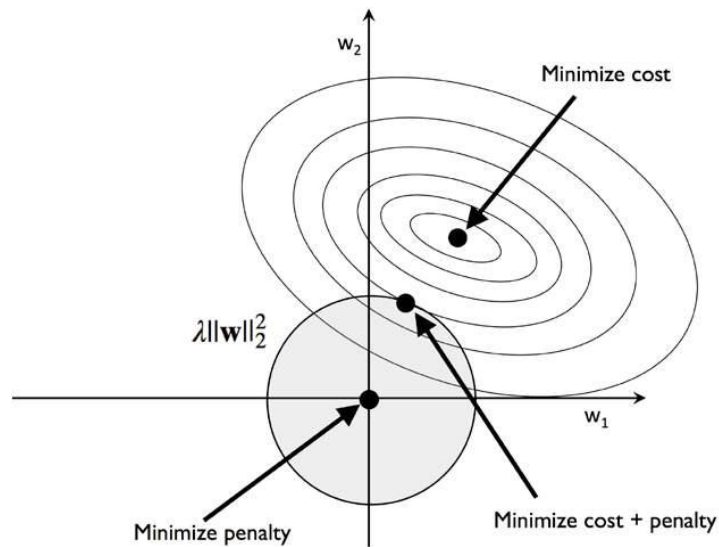
$$L1: \quad \|\mathbf{w}\|_1 = \sum_{j=1}^m |w_j|$$

SECTION 05

L2 vs. L1 Regularization — A Geometric View

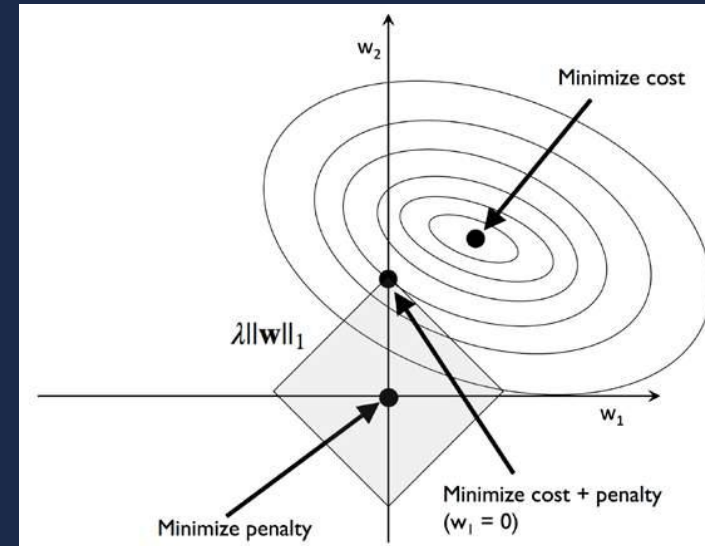
Regularization adds a penalty term to the cost function that shrinks weights toward zero. The shape of that penalty budget determines where the optimum lands.

L2 Regularization



Weights rarely land exactly on an axis — solutions have more non-zero than zero entries.

L1 Regularization

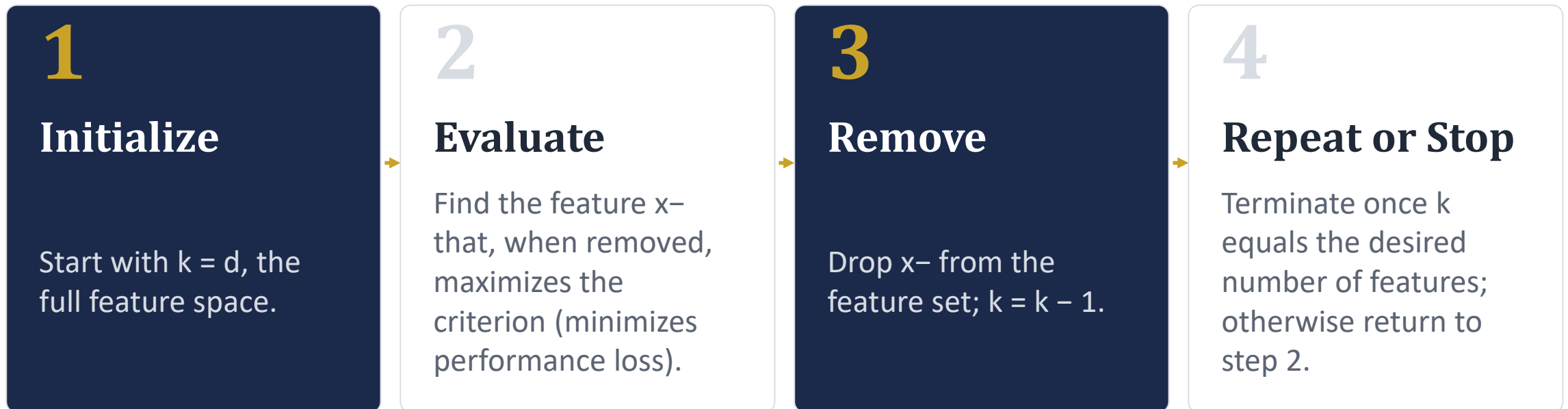


The sharp corners of the diamond make it far more likely the optimum lands on an axis — encouraging sparse solutions where many weights are exactly zero. This is how L1 regularization performs feature selection.

SECTION 05

Sequential Backward Selection (SBS)

- Another way to reduce the model complexity and avoid overfitting is **dimensionality reduction**
- A classic greedy search algorithm that reduces a d -dimensional feature space to a k -dimensional subspace ($k < d$), removing the feature that causes the least performance loss at each stage.

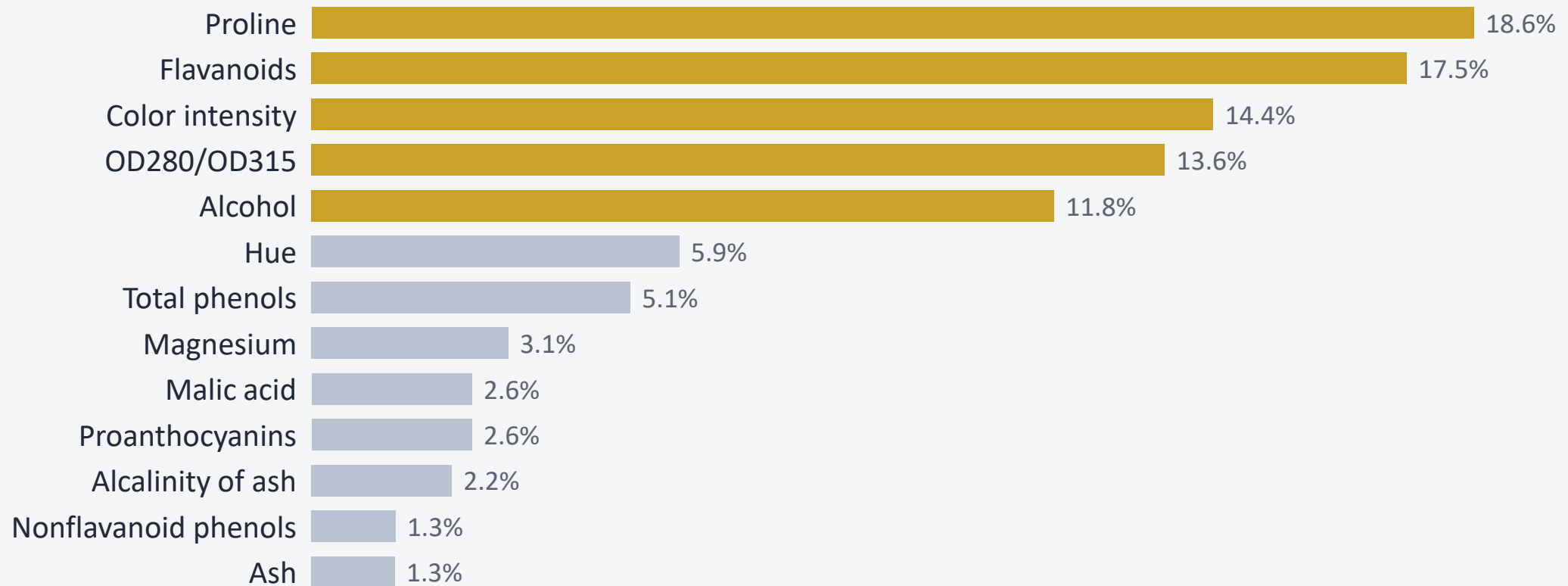


Applied to the Wine dataset with a KNN classifier, an SBS-selected 3-feature subset (Alcohol, Malic acid, OD280/OD315) reached ~92.6% test accuracy vs. ~96.3% with all 13 features.

SECTION 05

Feature Importance With Random Forests

Random forests measure feature importance as the averaged impurity decrease across all trees, without assuming the data is linearly separable — and no scaling or normalization is required.



Top 5 features (gold) account for over 75% of total impurity decrease across 500 trees.

CHAPTER SUMMARY

Building Good Training Datasets

- 1 Handle missing data deliberately — remove or impute, understanding the trade-offs of each.
- 2 Encode categorical variables correctly: ordinal features get ordered integers, nominal features get one-hot encoding.
- 3 Partition data into training and test sets to obtain an unbiased performance estimate.
- 4 Scale features — via normalization or standardization — for any algorithm that isn't tree-based.
- 5 Reduce dimensionality through L1 regularization, sequential feature selection, or random forest importance to fight overfitting.